

GCS-SAKER 개발 구조 변경 및 검증 보고서

작성일: 2026-06-01
대상: M2-M7 스트리밍/운영 안정화 개발 흐름

GCS-SAKER 개발 구조 변경 및 검증 보고서

작성일: 2026-06-01

1. 요약

GCS-SAKER는 초기 Python 백엔드와 React 대시보드 기반의 WebRTC 스트리밍 검증 구조에서 출발해, 현재는 Spring/Kotlin 인증/인가, Go media-control, MediaMTX, Redis, React/TypeScript 대시보드가 역할을 나눠 맡는 M7 점진 이전 구조로 발전했다.

이번 단계에서는 스트림 음성 수신 감지, 음성 활성화 스트림의 초록색 outline 표시, 사용자에게 불필요한 raw stream path 표시 제거, 세분화된 서버 상태 패널, 이벤트 로그/운영 지표 화면을 추가했다. 서버 컨테이너 로그는 Docker json-file 드라이버로 저장 중이며, Nginx edge 로그에서 /healthz, /readyz, /api/telemetry/all, /media-control/api/v1/streams 요청이 정상 기록되는 것을 확인했다.

2. 구조 변화 비교

2.1 초기 구조



특징:

- 빠른 MVP 검증에는 적합했지만 인증/인가, 미디어 제어, 운영 관측성이 한 시스템에 섞이기 쉬웠다.
- 브라우저가 백엔드 포트나 미디어 포트를 직접 바라보는 문제가 있었다.

2.2 M2 단일 443 인입 구조



```
+--> Auth API
+--> Backend API
+--> MediaMTX HLS/WHEP
+--> WebSocket Future Path
```

변경 이유:

- 외부 공개 포트를 443/tcp 중심으로 제한해 운영 보안성을 높이기 위해서다.
- 브라우저에서 localhost:8001 같은 직접 API 호출이 발생하던 문제를 Nginx reverse proxy 기준으로 정리했다.
- WebRTC ICE media 검증 후 직접 후보 실패 시 8189/udp/tcp를 조건부로 열어 실제 송수신을 확인했다.

2.3 M7 점진 이전 구조

```
Nginx Edge
|
+--> React/TypeScript Dashboard
+--> Spring/Kotlin Auth-Policy
+--> Go Media-Control
+--> Python Legacy Runtime
+--> MediaMTX
+--> Redis
+--> MySQL
```

변경 이유:

- 인증/인가 정책은 Spring/Kotlin으로 분리해 타입 안정성과 테스트 가능성을 높였다.
- MediaMTX 제어와 stream registry 조치는 Go media-control로 분리해 빠른 I/O와 명확한 API contract를 확보했다.
- Redis는 stream list cache와 refresh/session 계열 최적화 후보로 사용한다.
- Python legacy runtime은 기존 telemetry/control 기능의 호환 레이어로 유지한다.

2.4 현재 운영 관측성 구조

```
Dashboard
|
+--> Server Status Poller
|   +--> /healthz
|   +--> /readyz
|   +--> /media-control/healthz
|   +--> /api/v1/streams
|
+--> Event Log View
|   +--> severity / content / time range filters
|   +--> connections / RTT / throughput graph
|
+--> Realtime Player
|   +--> WHEP playback
|   +--> audio track detection
|   +--> active audio outline
```

3. 주요 테스트 중 발견한 문제와 수정

3.1 로그인 500 및 직접 백엔드 호출

문제:

- 로그인 요청이 `http://localhost:8001/auth/login`으로 나가며 운영 edge 경로를 타지 않았다.
- 서버 환경에서는 브라우저가 로컬 백엔드를 찾을 수 없어 500 또는 연결 실패가 발생했다.

수정:

- API 호출 기준을 Nginx reverse proxy 경로로 정리했다.
- 접속 가능한 표면은 edge가 담당하고, 내부 서비스는 compose network 안에서 라우팅되도록 했다.

3.2 WebRTC signaling은 되었지만 media가 오지 않던 문제

문제:

- WHEP signaling 응답은 왔지만 ICE media path가 완성되지 않아 화면이 뜨지 않았다.
- 외부 NAT 환경에서 직접 후보가 실패했고, media port 개방/relay 설정이 부족했다.

수정:

- STUN/TURN 구성을 점검하고, ICE 실패 시 relay 경로를 사용하도록 구성했다.
- 8189/udp/tcp 개방 후 실제 휴대폰 송출 영상이 대시보드에서 보이는 것을 확인했다.

3.3 MariaDB와 MySQL 포트 충돌

문제:

- 과거 MariaDB 컨테이너가 남아 포트를 선점하면서 MySQL이 정상 기동하지 못했다.
- 인증/세션/운영 데이터가 어느 DB에 쌓였는지 혼선이 생길 수 있었다.

수정:

- 불필요한 MariaDB 프로세스와 컨테이너를 정리했다.
- 현재 compose 기준 DB 구성을 MySQL 중심으로 고정했다.

3.4 stream registry empty array 처리

문제:

- `/media-control/api/v1/streams`가 빈 배열을 반환했을 때 기존 자산이 offline으로 반영되지 않았다.
- 송출이 끊겨도 자산트리와 카드가 연결 상태처럼 남았다.

수정:

- stream registry 응답이 빈 배열이면 기존 stream slot과 자산트리 항목을 offline으로 동기화하도록 수정했다.

3.5 GPS telemetry와 선택 스트림 동기화

문제:

- GPS telemetry는 수신되지만 선택 스트림 기준으로 지도 중심, telemetry panel, 자산 상태가 일관되게 갱신되지 않았다.

수정:

- 선택된 stream slot의 geometry를 기준으로 지도 focus와 지오메트리/텔레메트리 패널을 갱신하게 정리했다.
- 다른 stream pin은 유지하고 focus만 선택 스트림으로 이동하도록 했다.

3.6 음성 스트림 표시 부재

문제:

- WebRTC stream에 audio track이 포함되어도 대시보드 UI에는 표시되지 않았다.
- 어떤 stream에서 음성이 들어오는지 운영자가 즉시 알기 어려웠다.

수정:

- WebRTC remote stream의 audio track 상태를 감지한다.
- audio track이 live/enabled 상태이면 player, 선택 스트림, stream card에 초록색 outline과 음성 badge를 표시한다.
- 브라우저 autoplay 정책상 실제 음성 재생은 사용자 gesture가 필요할 수 있으므로 controls를 노출했다.

4. 이번 단계 구현 내용

4.1 스트림 음성 수신

- useWhepPlayback에서 remote stream audio track을 500ms 주기로 감지한다.
- WebRTCPlayer가 hasAudioTrack, isAudioActive 상태를 dashboard로 전달한다.
- active audio stream은 초록색 outline으로 강조된다.

4.2 사용자에게 불필요한 raw path 숨김

- stream path 또는 raw id 대신 표시명을 우선 노출한다.
- 연결 팝업에서 운영자가 표시 이름을 직접 입력할 수 있다.
- MAC 주소는 일반 브라우저에서 직접 읽을 수 없으므로, 장비 agent 또는 backend registry가 제공해야 한다.

4.3 서버 상태 세분화

- 기존의 단일 "서버상태" 대신 API 서버, 인증/인가 서버, Signaling 서버, Stream Registry, 네트워크 RTT, 통합 헬스체크로 분리했다.
- /healthz, /readyz, /media-control/healthz, /api/v1/streams를 주기적으로 확인한다.

4.4 이벤트 로그 화면

- 이벤트로그 탭을 추가했다.
- 내용, 시간 기간, severity 필터를 제공한다.
- connections, 평균 RTT, peak throughput, warn/error 개수를 요약한다.
- 시간대별 throughput/connection 그래프와 이벤트 테이블을 제공한다.

5. 검증 결과

5.1 프론트엔드 테스트

명령:

```
npm test -- --run
```

결과:

- 25개 test file 통과
- 126개 test 통과

5.2 프론트엔드 커버리지

명령:

```
npm run test:coverage
```

결과:

- Statements: 69.49%
- Branches: 54.47%
- Functions: 70.29%
- Lines: 70.31%

주요 파일:

- WebRTCPlayer.tsx: 100% statements / 100% lines
- useWhepPlayback.ts: 87.09% statements / 90.34% lines
- DashboardMvp.tsx: 87.23% statements / 86.90% lines
- operationalEvents.ts: 100% statements / 100% lines

5.3 빌드

명령:

```
npm run build
```

결과:

- TypeScript typecheck 통과
- Vite production build 통과

5.4 서버 로그 확인

확인 항목:

- edge

- media-control
- auth-policy
- mediamtx

결과:

- 주요 컨테이너가 Docker json-file 로그 드라이버를 사용 중이다.
- /healthz, /readyz, /api/telemetry/all, /media-control/api/v1/streams 요청이 edge access log에 기록된다.

6. 남은 리스크와 다음 작업

6.1 실제 서버 로그 ingestion

현재 이벤트로그 화면은 대시보드 운영 이벤트 모델과 필터 UI를 제공한다. 다음 단계에서는 Docker log 또는 edge access log를 수집하는 backend endpoint를 추가해 실제 서버 로그를 dashboard graph에 연결해야 한다.

권장 구현:

- edge access log parser
- media-control/auth-policy structured JSON log
- /api/v1/ops/events endpoint
- severity/category/source/time range server-side filtering

6.2 MAC 주소 기반 장비 식별

브라우저는 보안 정책상 클라이언트 MAC 주소를 직접 제공하지 않는다. 따라서 다음 중 하나가 필요하다.

- 송출장치 agent가 MAC 또는 device serial을 registration API로 전송
- Android native app이 device identifier를 안전하게 등록
- 운영자가 장비 등록 화면에서 MAC/serial을 사전 등록

6.3 음성 autoplay 정책

브라우저는 사용자 gesture 없는 audio 재생을 막을 수 있다. 대시보드에서는 controls와 active audio indicator를 제공하되, 운영 UX에서는 “음성 켜기” 명령을 명확히 두는 것이 안전적이다.

6.4 폐쇄망 운영

폐쇄망 납품을 위해 다음 항목이 계속 필요하다.

- 자체 STUN/TURN 이중화
- 외부 지도 API 비의존 offline map tile
- 내부 NTP/time API
- npm/gradle/go module offline mirror 또는 사전 vendor bundle
- Docker가 없는 환경을 위한 설치 패키지 전략

7. 결론

현재 GCS-SAKER는 단순 스트리밍 MVP에서 운영 가능한 저지연 스트리밍 관제 플랫폼 구조로 이동 중이다. 이번 변경으로 운영자가 영상뿐 아니라 음성 활성 상태, 세분화된 서버 상태, 이벤트/네트워크 지표를 한 화면에서 확인할 수 있는 기반이 마련되었다. 다음 우선순위는 실제 서버 로그 ingestion API와 장비 registry 기반 MAC/serial 식별, 그리고 폐쇄망 운영 패키징이다.